

Digital Forensics Incident Report

Memory Forensics Analysis – Compromised XAMPP/DVWA Host

Prepared By: Dnyaneshwari Kale

Capstone Challenge 4

Tool: Volatility 2.6 (profile Win2008SP1x86 / VistaSP1x86)

Memory image captured: 2015-09-03 10:04:05 UTC

Report prepared: August 2026

Executive Summary

This report documents the forensic examination of a 1 GB Windows memory image (memdump.mem) acquired from a XAMPP-based web server hosting the Damn Vulnerable Web Application (DVWA). Analysis was performed using Volatility 2.6 against a Windows Vista SP1 / Server 2008 SP1 (x86) memory profile.

The evidence establishes that an attacker at 192.168.56.102, operating on the same internal/host-only network as the victim (192.168.56.101), conducted reconnaissance via directory traversal (Local File Inclusion), followed by manual and then automated (SQLMap) SQL injection against the DVWA application. The attacker subsequently obtained a PHP web shell (c99.php) in the DVWA web root and gained interactive command-line access as Administrator, which was used to create two backdoor local accounts, grant Remote Desktop access, and enable RDP through the Windows Firewall. Memory analysis further identified obfuscated shellcode injected into multiple system processes, consistent with a process-injection/migration technique.

This is a controlled/lab environment (VirtualBox host-only network, DVWA training application) rather than a live production breach, which is reflected in the hypothesis and conclusions below.

Q1. What type of attacks has been performed on the box?

The attacker carried out a multi-stage web application compromise followed by post-exploitation persistence activity. Apache access-log entries recovered from memory (via dumpfiles) confirm the following attack chain, all originating from 192.168.56.102:

- Reconnaissance of the DVWA file-upload feature (2015-09-02 01:43 UTC-7).
- Local File Inclusion / directory traversal via the DVWA 'fi' (file inclusion) module, used to read Windows and application configuration files including hosts, an administrator data file, and phpMyAdmin configuration (02:31–02:38 UTC-7).
- Manual SQL injection against the DVWA 'sqli' module using UNION-based payloads to enumerate the database version, current user, database name, tables, and columns (03:58–04:15 UTC-7).
- Automated SQL injection using SQLMap (User-Agent: sqlmap/1.0-dev-nongit-20150902) running boolean-based, UNION-based, error-based, and time-based injection tests (04:15–04:18 UTC-7).
- Deployment of a PHP web shell (c99.php) into the DVWA web root, most likely via the file-upload vulnerability that was reconnoitred earlier (upload mechanics are not visible in the access log because Apache's combined log format does not record multipart file contents).
- Command execution as Administrator via an interactive console (confirmed via cmdscan), used for account-creation and Remote Desktop persistence (see Q2).

Evidence

Evidence	Meaning
GET /dvwa/vulnerabilities/fi/?page=../../../../​/windows/system32/drivers/etc/hosts	Local File Inclusion / directory traversal
GET /dvwa/vulnerabilities/sqli/?id=a' union select ... --	Manual SQL injection
User-Agent: sqlmap/1.0-dev-nongit-20150902	Automated SQL injection (SQLMap)
\\xampp\htdocs\DVWA\c99.php (filescan offset 0x3ee2e120)	PHP web shell deployed in web root
net user user1 ... /add ; net localgroup "Remote Desktop Users" user1 /add	Backdoor account + RDP persistence
netsh firewall set service type=remotedesktop mode=enable scope=subnet	RDP opened through Windows Firewall

```

PS C:\Users\hp\OneDrive\Documents\Desktop\Capstone_Challenges\TEST 1> Select-String -Path ".\WORK\output\filescan.txt" -Pattern "c99.php|webshells.zip"

WORK\output\filescan.txt:2009:0x00000003ee2e120      4      0 R--rw- \Device\HarddiskVolume1\xampp\htdocs\DVWA\c99.php
WORK\output\filescan.txt:2351:0x00000003eeabba0      8      0 RW---- \Device\HarddiskVolume1\xampp\htdocs\DVWA\webshells.zip

PS C:\Users\hp\OneDrive\Documents\Desktop\Capstone_Challenges\TEST 1> Get-Content ".\WORK\output\cmdscan.txt" |
>> Select-String -Pattern "net user|localgroup|netsh firewall|remotedesktop"

Cmd #3 @ 0x5a34bd0: net user user1 user1 /add
Cmd #4 @ 0x5a34eb8: net user user1 root@psut /add
Cmd #5 @ 0x5a34c10: net user user1 Root@psut /add
Cmd #8 @ 0x5a34d88: net localgroup /?
Cmd #9 @ 0x5a34f48: net localgroup "Remote Desktop Users" user1 /add
Cmd #12 @ 0xe907e8: netsh firewall /?
Cmd #13 @ 0xe91218: netsh firewall set service type = remotedesktop /?
Cmd #14 @ 0xe91288: netsh firewall set service type = remotedesktop enable
Cmd #15 @ 0xe91300: netsh firewall set service type=remotedesktop mode=enable
Cmd #16 @ 0xe91380: netsh firewall set service type=remotedesktop mode=enable scope=subnet
Cmd #0 @ 0xe91970: netsh firewall set service type=remotedesktop mode=enable scope=subnet
Cmd #1 @ 0x5a17b58: netsh firewall set service type=remotedesktop mode=enable scope=subnet

```

Figure 1. Filtered cmdscan output showing account-creation and RDP-enablement commands, alongside filescan confirmation of c99.php and webshells.zip in the DVWA web root.

Q2. How many users has the attacker(s) added to the box, and how were they added?

Two accounts were added: user1 and hacker. Both are confirmed present via hashdump against the recovered SAM hive, and both share an identical NTLM hash, indicating the same password was used for each.

user1

Creation is directly evidenced in the recovered command history (cmdscan): the attacker made three attempts (net user user1 user1 /add, net user user1 root@psut /add, net user user1 Root@psut /add) before the account was successfully created, then added it to the Remote Desktop Users group with net localgroup "Remote Desktop Users" user1 /add, which the console output confirms completed successfully.

hacker

No net user hacker ... /add command was recovered in the console command-history buffers. The account's existence is confirmed only via hashdump. This is most likely explained by circular buffer reuse in the console history structures Volatility recovered — the buffers examined show clear signs of being overwritten (fragmented remnants such as 'et.exe', a truncated remainder of net.exe, were observed at higher slot indices than the commands actually recovered). The creation method for the hacker account therefore cannot be attributed to a specific recovered command, though the identical password hash and adjacent SAM RID (1006, immediately following user1's 1005) strongly suggest it was created via the same method, in the same session.

```

PS C:\Users\hp\OneDrive\Documents\Desktop\Capstone_Challenges\TEST 1> .\volatility_2.6_win64_standalone.exe -f ".\WORK\memory\memdump.mem" --profile=Win2008SP1x86 hashdump
Volatility Foundation Volatility Framework 2.6
Administrator:500:aad3b435b51404eeaad3b435b51404ee:63d6a39b8467b94ae92ab1931d4079dd:::
Guest:501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
user1:1005:aad3b435b51404eeaad3b435b51404ee:817875ce4794a9262159186413772644:::
hacker:1006:aad3b435b51404eeaad3b435b51404ee:817875ce4794a9262159186413772644:::
PS C:\Users\hp\OneDrive\Documents\Desktop\Capstone_Challenges\TEST 1>

```

Figure 2. hashdump output confirming both user1 (RID 1005) and hacker (RID 1006) with identical NTLM hashes.

```

Windows PowerShell
PS C:\Users\hnp\OneDrive\Documents\Desktop\Capstone_Challenges\TEST 1> .\volatility_2.6_win64_standalone.exe -f ".\WORK\memory\memdump
.mem" --profile=Win2008SP1x86 cmdscan
Volatility Foundation Volatility Framework 2.6
*****
CommandProcess: csrss.exe Pid: 524
CommandHistory: 0x5a24708 Application: cmd.exe Flags: Allocated, Reset
CommandCount: 17 LastAdded: 16 LastDisplayed: 16
FirstCommand: 0 CommandCountMax: 50
ProcessHandle: 0x2d8
Cmd #0 @ 0xe907c8: ipconfig
Cmd #1 @ 0xe91af8: cls
Cmd #2 @ 0xe91db0: ipconfig
Cmd #3 @ 0x5a34bd0: net user user1 user1 /add
Cmd #4 @ 0x5a34eb8: net user user1 root@psut /add
Cmd #5 @ 0x5a34c10: net user user1 Root@psut /add
Cmd #6 @ 0x5a24800: cls
Cmd #7 @ 0x5a34c58: net /?
Cmd #8 @ 0x5a34d88: net localgroup /?
Cmd #9 @ 0x5a34f48: net localgroup "Remote Desktop Users" user1 /add
Cmd #10 @ 0x5a34c70: net /?
Cmd #11 @ 0xe911b0: netsh /?
Cmd #12 @ 0xe907e8: netsh firewall /?
Cmd #13 @ 0xe91218: netsh firewall set service type = remotedesktop /?
Cmd #14 @ 0xe91288: netsh firewall set service type = remotedesktop enable
Cmd #15 @ 0xe91300: netsh firewall set service type=remotedesktop mode=enable
Cmd #16 @ 0xe91380: netsh firewall set service type=remotedesktop mode=enable scope=subnet
*****
CommandProcess: csrss.exe Pid: 524
CommandHistory: 0x5a30950 Application: cmd.exe Flags: Allocated, Reset
CommandCount: 2 LastAdded: 1 LastDisplayed: 1
FirstCommand: 0 CommandCountMax: 50
ProcessHandle: 0x7ec
Cmd #0 @ 0xe91970: netsh fireall set service type=remotedesktop mode=enable scope=subnet

```

Figure 3. cmdscan output showing the recovered net user / net localgroup / netsh firewall commands for user1. No equivalent net user command was recovered for hacker.

Q3. What leftovers (files, tools, info, etc.) did the attacker(s) leave behind?

The following attacker artifacts were recovered directly from the memory image:

Leftover	Evidence / Notes
c99.php	PHP web shell placed at \xampp\htdocs\DVWA\c99.php — abnormal location for a stock DVWA install
webshells.zip	Archive of additional web-shell tooling, alongside c99.php in the DVWA root
Command history (cmdscan)	net user / net localgroup / netsh firewall commands recovered directly from console buffers
ad_driver.sys	Unexplained driver file in Administrator's Temp directory; no corresponding install/execution command was recovered, so attribution to the attacker is unconfirmed
install.exe	Unexplained executable at the drive root (C:\install.exe); origin and purpose not established from available evidence
PHP session files (xampp\tmp\sess_*)	Confirms active, authenticated browsing sessions against the DVWA application around the time of the attack

```
Windows PowerShell
PS C:\Users\hp\OneDrive\Documents\Desktop\Capstone_Challenges\TEST 1> Select-String -Path ".\WORK\output\filescan.txt" -Pattern "\.php"

WORK\output\filescan.txt:924:0x00000000181ce68      8      0 R--rw-
\Device\HarddiskVolume1\xampp\htdocs\DVWA\vulnerabilities\xss_r\index.php
WORK\output\filescan.txt:1204:0x000000001efd71e8      8      0 R--rw-
\Device\HarddiskVolume1\xampp\htdocs\DVWA\vulnerabilities\xss_r\source\low.php
WORK\output\filescan.txt:1467:0x0000000026585238      8      0 R--rw-
\Device\HarddiskVolume1\xampp\htdocs\DVWA\vulnerabilities\upload\source\low.php
WORK\output\filescan.txt:1738:0x000000002a63d4b8      8      0 R--rw-
\Device\HarddiskVolume1\xampp\htdocs\DVWA\vulnerabilities\sql_i\index.php
WORK\output\filescan.txt:1929:0x000000003ee0ca30      8      0 R--rw-
\Device\HarddiskVolume1\xampp\htdocs\DVWA\vulnerabilities\xss_r\source\medium.php
WORK\output\filescan.txt:2009:0x000000003ee2e120      4      0 R--rw- \Device\HarddiskVolume1\xampp\htdocs\DVWA\c99.php
WORK\output\filescan.txt:2011:0x000000003ee2f450      8      0 R--rw-
\Device\HarddiskVolume1\xampp\htdocs\DVWA\config\config.inc.php
WORK\output\filescan.txt:2092:0x000000003ee4b168      8      0 R--rw-
\Device\HarddiskVolume1\xampp\htdocs\DVWA\vulnerabilities\xss_r\index.php
WORK\output\filescan.txt:2328:0x000000003ee9e8b8      8      0 R--rw-
\Device\HarddiskVolume1\xampp\htdocs\DVWA\vulnerabilities\sql_i\source\medium.php
WORK\output\filescan.txt:2334:0x000000003eea0388      8      0 R--rw-
\Device\HarddiskVolume1\xampp\htdocs\DVWA\vulnerabilities\sql_i\source\high.php
WORK\output\filescan.txt:3023:0x000000003ef5a3b0      8      0 R--rw- \Device\HarddiskVolume1\xampp\htdocs\DVWA\login.php
WORK\output\filescan.txt:3042:0x000000003ef5f8e0      8      0 R--rw-
\Device\HarddiskVolume1\xampp\htdocs\DVWA\vulnerabilities\upload\index.php
WORK\output\filescan.txt:3131:0x000000003ef760e8      8      0 R--rw- \Device\HarddiskVolume1\xampp\htdocs\DVWA\setup.php
WORK\output\filescan.txt:3565:0x000000003f1f4f80      8      0 R--rw-
\Device\HarddiskVolume1\xampp\htdocs\DVWA\vulnerabilities\xss_r\source\high.php
WORK\output\filescan.txt:3666:0x000000003f2284f8      8      0 R--rw-
\Device\HarddiskVolume1\xampp\htdocs\DVWA\vulnerabilities\sql_i\source\low.php
WORK\output\filescan.txt:3788:0x000000003f271888      8      0 R--rw-
\Device\HarddiskVolume1\xampp\htdocs\DVWA\includes\DBMS\MySQL.php
```

Figure 4. filescan output showing c99.php and the surrounding DVWA vulnerability module files recovered from the memory image.

Note on scope: the memory image alone does not conclusively show a modified c99.php banner (e.g. an “Owned by Hacker” string) or a data.txt file containing attacker-supplied content. If such artifacts exist, they would need to be confirmed via disk/file-content examination rather than the memory-resident filescan evidence used here, and are flagged as outstanding items (see Limitations, below).

Q4. What software has been installed on the box, and were they installed by the attacker(s) or not?

Pre-existing / not attacker-installed:

XAMPP (Apache httpd.exe, MySQL mysqld.exe, FileZilla Server) and the DVWA training application were already present and running at the time of the earliest recorded process-creation timestamps (2015-08-23), well before any attacker activity in the logs (2015-09-02 onward). These form the intentionally vulnerable lab environment the attacker exploited, not attacker-installed tooling.

FTK Imager (running from a VirtualBox shared folder, \\Vboxsvr\101\FTK-Imager\)) was launched at 2015-09-03 10:03:37, one minute before the memory capture — this is the investigation team's own acquisition tool, not attacker activity.

Attacker-introduced:

The only software artifacts attributable to the attacker are the c99.php web shell and webshells.zip archive placed in the DVWA web root. No evidence was found of a separately installed application (e.g. an MSI/EXE installer or new Windows service) attributable to the attacker.

ad_driver.sys, found in the Administrator's Temp directory, could not be attributed to the attacker or to legitimate software with the available evidence — no install, load, or execution command referencing this file was recovered. It is documented as an unresolved artifact rather than a confirmed attacker tool.

```

Windows PowerShell
PS C:\Users\hnp\OneDrive\Documents\Desktop\Capstone_Challenges\TEST 1> Get-Content ".\WORK\output\cmdscan.txt" |
>> Select-String -Pattern "net user|localgroup|netsh firewall|remotedesktop"

Cmd #3 @ 0x5a34bd0: net user user1 user1 /add
Cmd #4 @ 0x5a34eb8: net user user1 root@psut /add
Cmd #5 @ 0x5a34c10: net user user1 Root@psut /add
Cmd #8 @ 0x5a34d88: net localgroup /?
Cmd #9 @ 0x5a34f48: net localgroup "Remote Desktop Users" user1 /add
Cmd #12 @ 0xe907e8: netsh firewall /?
Cmd #13 @ 0xe91218: netsh firewall set service type = remotedesktop /?
Cmd #14 @ 0xe91288: netsh firewall set service type = remotedesktop enable
Cmd #15 @ 0xe91300: netsh firewall set service type=remotedesktop mode=enable
Cmd #16 @ 0xe91380: netsh firewall set service type=remotedesktop mode=enable scope=subnet
Cmd #0 @ 0xe91970: netsh firewall set service type=remotedesktop mode=enable scope=subnet
Cmd #1 @ 0x5a17b58: netsh firewall set service type=remotedesktop mode=enable scope=subnet

PS C:\Users\hnp\OneDrive\Documents\Desktop\Capstone_Challenges\TEST 1> Get-Content ".\WORK\output\pstree.txt" |
>> Select-String -Pattern "xampp|httpd|mysql|FileZilla|FTK|cmd.exe|powershell"

. 0x83f68300:FTK Imager.exe                2120    816    13    382 2015-09-03 10:03:37 UTC+0000
. 0x83faa020:xampp-control.e             2768    816     2    119 2015-08-23 10:32:17 UTC+0000
.. 0x83e4d7c0:httpd.exe                  2796   2768     1     92 2015-08-23 10:32:21 UTC+0000
... 0x83fd77a8:httpd.exe                 2880   2796    155   483 2015-08-23 10:32:26 UTC+0000
.. 0x83fd5200:FileZillaServer            2856   2768     5     35 2015-08-23 10:32:25 UTC+0000
.. 0x83f9ec70:mysql.exe                  2804   2768    23   570 2015-08-23 10:32:23 UTC+0000
. 0x83e7b7f8:cmd.exe                     612    816     1     72 2015-08-23 10:30:44 UTC+0000
. 0x84259100:cmd.exe                     1972    816     1     19 2015-09-02 09:28:30 UTC+0000

```

Figure 5. Filtered pstree output confirming the pre-existing XAMPP process family and the investigator's FTK Imager process.

Q5. Using memory forensics, can you identify the type of shellcode used?

malfind identified five regions of private, executable, writable (PAGE_EXECUTE_READWRITE) memory with no backing file across four processes: svchost.exe (PID 1024), svchost.exe (PID 1108), explorer.exe (two regions), and xampp-control.exe. An identical byte pattern was found at the start of the svchost.exe (PID 1024), svchost.exe (PID 1108), and one explorer.exe region — a sequence of MOV AL,N / JMP instructions that all redirect to a common offset (region start + 0x74).

This MOV/JMP jump-table structure is a known obfuscation technique used to defeat naive linear disassembly and static signature scanning by hiding the real entry point of the payload behind an artificial branch table. Its presence, byte-for-byte identical, in two separate svchost.exe processes and in explorer.exe is consistent with process injection and code migration — a technique commonly associated with post-exploitation frameworks that copy their implant into other running processes for persistence.

Binary string extraction from the dumped regions returned no readable text (no plaintext API names, IP addresses, or framework identifiers), which is expected behaviour for shellcode that resolves Windows APIs at runtime via hash-based lookup rather than storing plaintext function names.

YARA scanning of all recovered outputs for known Metasploit/Meterpreter indicators (meterpreter, PAYLOAD_UUID, core_loadlib, core_machine_id, core_uuid) returned no matches. Framework attribution therefore cannot be confirmed from string evidence. The shellcode was not found within the Apache process (httpd.exe), indicating the injected code operates independently of the PHP-based web-shell access vector rather than being a PHP-hosted payload.

Conclusion:

The recovered code is consistent with obfuscated/encoded native x86 shellcode, injected via process injection and migrated across multiple system processes (svchost.exe ×2, explorer.exe). This is consistent with, but not conclusively attributable to, a Metasploit/Meterpreter-style stager, since no framework-specific strings were recovered. It is not evidence of a “PHP Meterpreter” payload, as the injected code is native (x86) and resides outside the Apache/PHP process space.

```

Windows PowerShell
PS C:\Users\hnp\OneDrive\Documents\Desktop\Capstone_Challenges\TEST 1> .\volatility_2.6_win64_standalone.exe -f ".\WORK\memory\memdump
.mem" --profile=Win2008SP1x86 malfind > ".\WORK\output\malfind.txt"
Volatility Foundation Volatility Framework 2.6
PS C:\Users\hnp\OneDrive\Documents\Desktop\Capstone_Challenges\TEST 1> type ".\WORK\output\malfind.txt"
Process: svchost.exe Pid: 1024 Address: 0xe60000
Vad Tag: VadS Protection: PAGE_EXECUTE_READWRITE
Flags: CommitCharge: 2, MemCommit: 1, PrivateMemory: 1, Protection: 6

0x00e60000 b0 00 eb 70 b0 01 eb 6c b0 02 eb 68 b0 03 eb 64 ...p...l...h...d
0x00e60010 b0 04 eb 60 b0 05 eb 5c b0 06 eb 58 b0 07 eb 54 ...'....\...X...T
0x00e60020 b0 08 eb 50 b0 09 eb 4c b0 0a eb 48 b0 0b eb 44 ...P...L...H...D
0x00e60030 b0 0c eb 40 b0 0d eb 3c b0 0e eb 38 b0 0f eb 34 ...@...<...8...4

0x00e60000 b000      MOV AL, 0x0
0x00e60002 eb70      JMP 0xe60074
0x00e60004 b001      MOV AL, 0x1
0x00e60006 eb6c      JMP 0xe60074
0x00e60008 b002      MOV AL, 0x2
0x00e6000a eb68      JMP 0xe60074
0x00e6000c b003      MOV AL, 0x3
0x00e6000e eb64      JMP 0xe60074
0x00e60010 b004      MOV AL, 0x4
0x00e60012 eb60      JMP 0xe60074
0x00e60014 b005      MOV AL, 0x5
0x00e60016 eb5c      JMP 0xe60074
0x00e60018 b006      MOV AL, 0x6
0x00e6001a eb58      JMP 0xe60074
0x00e6001c b007      MOV AL, 0x7
0x00e6001e eb54      JMP 0xe60074
0x00e60020 b008      MOV AL, 0x8
0x00e60022 eb50      JMP 0xe60074
0x00e60024 b009      MOV AL, 0x9
0x00e60026 eb4c      JMP 0xe60074

```

Figure 6. malfind output for svchost.exe (PID 1024) showing the PAGE_EXECUTE_READWRITE region and the MOV/JMP jump-table obfuscation stub, identical to the region recovered from svchost.exe PID 1108 and explorer.exe.

Q6. What is the timeline analysis for all events that happened on the box?

UTC Timestamp	Event	Evidence Source
2015-08-23 22:41 UTC	XAMPP environment installed	Filesystem timeline
2015-08-23 22:52 UTC	DVWA installed under /xampp/htdocs/DVWA	Filesystem timeline
2015-09-02 01:43 UTC-7	Reconnaissance of file-upload page (192.168.56.102)	Apache access.log
2015-09-02 02:31–02:38 UTC-7	LFI / directory-traversal probing (/etc/hosts, data.txt, phpMyAdmin config)	Apache access.log
2015-09-02 03:58–04:15 UTC-7	Manual SQL injection (UNION-based enumeration of DB, tables, users)	Apache access.log
2015-09-02 04:15–04:18 UTC-7	Automated SQL injection using SQLMap 1.0-dev-nongit-20150902	Apache access.log (User-Agent)
2015-09-02 04:25:52 UTC-7	Final probing request from attacker IP (denied)	Apache error.log
2015-09-02 09:05:06 UTC-7	User 'user1' created, added to Remote Desktop Users	SAM hashdump + cmdscan
2015-09-02 09:05:25 UTC-7	User 'hacker' created (creation command not recovered)	SAM hashdump
2015-09-02 23:52:24 UTC-7	Second probing request from attacker IP (denied)	Apache error.log
2015-09-03 (exact time not confirmed)	c99.php webshell present in DVWA web root	filescan / dumped file
2015-09-03 10:03:06 UTC	TrustedInstaller.exe activity (last pre-	ps tree

UTC Timestamp	Event	Evidence Source
	capture system event)	
2015-09-03 10:03:37 UTC	FTK Imager launched (investigator acquisition, not attacker)	pstree / cmdline
2015-09-03 10:04:05 UTC	Memory image captured	imageinfo

```

PS C:\Users\hp\OneDrive\Documents\Desktop\Capstone_Challenges\TEST 1> $timeline | Format-Table -AutoSize
UTC          Event                                     Evidence
----          -
2015-08-23 22:41 XAMPP environment installed             Filesystem timeline: /xampp created
2015-08-23 22:52 DVWA installed             Filesystem timeline: /xampp/htdocs/DVWA created
2015-09-02 09:05:06 User 'user1' created   SAM/account timestamp + net user activity
2015-09-02 09:05:25 User 'hacker' created   SAM/account timestamp
2015-09-02 09:32 data.txt created containing 'hello'   Filesystem artifact
2015-09-03 06:52 SQL injection / PHP upload activity   Apache access-log analysis
2015-09-03 07:16 phpshell.php accessed           Apache access-log analysis
2015-09-03 07:17 Directory 'abc' created           Webshell command execution
2015-09-03 07:20 c99.php modified; 'Owned by Hacker' added   Filesystem/file-content analysis
2015-09-03 10:03 FTK Imager started           pslist/pstree memory evidence

```

Figure 7. Consolidated timeline as compiled during analysis.

Evidence gap: no web-application requests from 192.168.56.102 were recovered between the final probing request at 04:25:52 (denied, per error.log) and the account-creation events at 09:05:06–09:05:25. This is either a gap in the recovered log coverage (Apache's cached log data in memory is partial, not a complete rotation-to-rotation record) or reflects the attacker transitioning to a non-HTTP-logged access channel — consistent with the interactive “Administrator: Command Prompt” console session recovered via cmdscan, which implies the attacker already had shell-level access by the time the accounts were created. This gap is stated explicitly rather than bridged with unsupported assumptions.

Q7. What is your hypothesis for the case, and what is your approach in solving it?

Hypothesis

The victim machine was a XAMPP/DVWA lab host on an internal VirtualBox host-only network (192.168.56.0/24). An attacker at 192.168.56.102, on the same network segment, exploited the intentionally vulnerable DVWA application: first reconnoitring the file-upload and file-inclusion modules, then extracting database contents via manual and automated (SQLMap) SQL injection, then deploying a PHP web shell (c99.php) into the DVWA web root — most likely via the file-upload vulnerability identified during reconnaissance. Using shell-level command execution as Administrator, the attacker created two backdoor accounts (user1, hacker), granted Remote Desktop access, and opened RDP through the Windows Firewall to establish persistent remote access. Separately, obfuscated native shellcode was found injected into multiple Windows system processes, indicating an additional post-exploitation stage involving process injection, though this could not be conclusively attributed to a specific framework.

Approach

The investigation combined memory forensics (Volatility 2.6 against a Win2008SP1x86/VistaSP1x86 profile) with web-server log analysis extracted from the same memory image. Process listing and reconstruction (pslist, psscan, pstree) established the running software baseline. Command-history recovery (cmdscan, consoles) and registry analysis (hashdump against the recovered SAM hive) established account-creation and persistence activity. Filesystem enumeration (filesan) located attacker-left files including the web shell and its archive. Memory-resident code-injection detection (malfind) combined with binary string extraction and YARA scanning

characterised the injected shellcode. Finally, dumpfiles was used to recover the cached Apache access.log and error.log directly from memory, which supplied the source IP, exact exploitation timeline, and attack technique (LFI, manual SQLi, SQLMap) that anchor the rest of the findings. All conclusions were cross-referenced against timestamps to build a single, internally consistent timeline, and claims not supported by recovered evidence (e.g. unconfirmed shellcode framework attribution, the account-creation access channel) are explicitly flagged as such rather than asserted.

Q8. Is there anything else you would like to add?

This engagement illustrates the value of cross-referencing memory artifacts with cached log data recoverable from the same image — the Apache access.log and error.log, both recovered via dumpfiles, transformed several previously unsupported claims (attacker source IP, exact exploitation technique and timing) into directly cited evidence, and also surfaced an honest evidence gap (the account-creation access channel) that would otherwise have been glossed over.

A related lesson: the presence of a suspicious file (e.g. ad_driver.sys, install.exe) is not, by itself, proof that the attacker installed it. Both remain undetermined in this report because no corroborating execution, installation, or command-history evidence was recovered — they are documented as open items rather than attributed.

Finally, source IP evidence places the attacker on the same internal/host-only network as the victim, which is consistent with this being a controlled lab/training exercise rather than an internet-facing breach; this framing should be reflected in any remediation recommendations (e.g. network segmentation would not have been a relevant control here, whereas patching/removing DVWA, input validation, and file-upload restrictions would be).

Outstanding Evidence and Limitations

The following items are referenced in earlier drafts of this investigation but are not yet confirmed by evidence captured in this report. They should either be independently verified or removed from the final submission:

- Direct confirmation of the c99.php upload request (Apache's combined log format does not record multipart POST file contents, so the exact upload moment is inferred, not logged).
- A modified c99.php banner or “Owned by Hacker” string — not observed in the memory-resident files can copy examined; would require file-content extraction and review.
- A data.txt file containing attacker-supplied content (e.g. “hello”) — not located during files can/Apache log review in this report.
- The access channel used to create the user1/hacker accounts at 09:05 (web shell continuation vs. RDP vs. console) — stated as an evidence gap in Q6 rather than resolved.
- Definitive framework attribution for the injected shellcode (Q5) — YARA scanning for Meterpreter-specific strings returned no matches; the classification given is behavioural (injection/migration pattern), not signature-confirmed.
- Attribution of ad_driver.sys and install.exe to either the attacker or legitimate software.